

INSTITUTO FEDERAL
ESPÍRITO SANTO
Campus Cachoeiro de Itapemirim

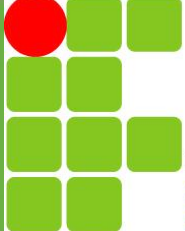
Programação I

INTRODUÇÃO A LINGUAGEM C PONTEIRO E MODULARIZAÇÃO

Prof. Flávio Izo | fizo@ifes.edu.br

Ministério
da Educação

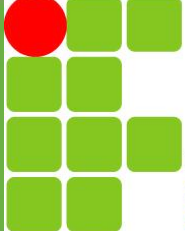




INSTITUTO FEDERAL
ESPÍRITO SANTO
Campus Cachoeiro de Itapemirim

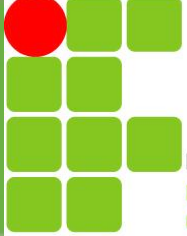
Agenda

- ▶ 1.0 Introdução a Ponteiros
- ▶ 2.0 Vetores e Ponteiros
- ▶ 3.0 Modularização
- ▶ 4.0 Treinando



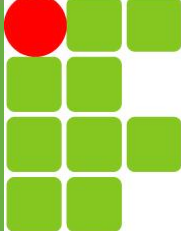
INSTITUTO FEDERAL
ESPÍRITO SANTO
Campus Cachoeiro de Itapemirim

INTRODUÇÃO A PONTEIROS



1.0 Introdução a Ponteiros

- ▶ Ponteiro é um tipo de variável cujo valor se refere diretamente a outro valor alocado em outra área da memória, através de seu endereço. Ponteiro é uma simples implementação do tipo referência.
- ▶ & Referencia um endereço de memória
- ▶ * Referencia o conteúdo de um endereço de memória



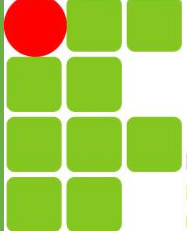
1.0 Introdução a Ponteiros

► Declaração e uso de um ponteiro.

```
modularizao09.c  X  lista08-exercicio08.c  X  ponteiro01.c  X
1  #include<stdio.h>
2
3  int main(){
4      int a, *b;
5
6      // & Referencia um endereço de memória
7      // * Referencia o conteúdo de um endereço de memória
8
9
10     // B recebe o endereço de memória da variável A
11     b = &a;
12     // O valor pra quem B está apontando recebe 10
13     *b = 10;
14
15
16     printf("Valor de A: %d \n", a);
17     printf("Valor de B: %d \n", b);
18
19     return 0;
20 }
21
```

```
C:\materialAu
Valor de A: 10
Valor de B: 2358868

Process returned 0 (0x0)
Press any key to continue.
```

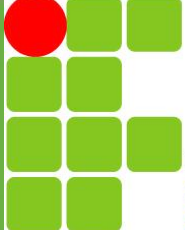


1.0 Introdução a Ponteiros

```
teiro02.c X
1  #include<stdio.h>
2
3  int main(){
4      int a, *b;
5
6      // & Referencia um endereço de memória
7      // * Referencia o conteúdo de um endereço de memória
8
9
10     a = 50;
11     printf("Valor anterior de A: %d \n", a);
12     // B recebe o endereço de memória da variável A
13     b = &a;
14     // O valor pra quem B está apontando recebe 10
15     *b = 10;
16
17
18     printf("Valor de A: %d \n", a);
19     printf("Valor de B: %d \n", b);
20     printf("Valor da variável apontada por B: %d \n", *b);
21
22     return 0;
23 }
```

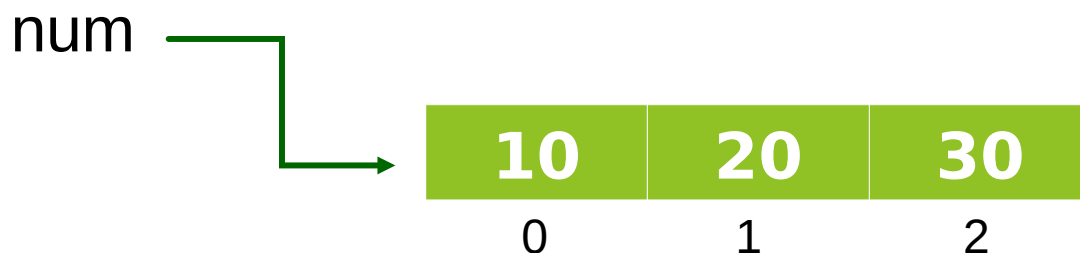
```
Valor anterior de A: 50
Valor de A: 10
Valor de B: 2358868
Valor da variável apontada por B: 10

Process returned 0 (0x0)   execution time = 0.000 sec(s)
Press any key to continue.
```



2.0 Vetores e Ponteiros

- De modo geral, vetores são ponteiros. Veja:



Resumindo:

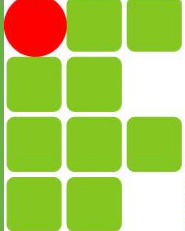
num = vetor de 3 posições

num contém o endereço de memória de num[0]

*num = num[0]

*(num+1) = num[1]

*(num+2) = num[2]



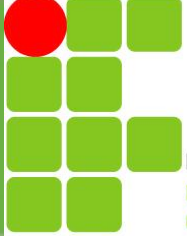
2.0 Vetores e Ponteiros

- Provando que `num` = endereço de memória de `num[0]`

```
ponteiro03.c X
1  #include<stdio.h>
2
3  int main(){
4      int num[3];
5
6      // & Referencia um endereço de memória
7      // * Referencia o conteúdo de um endereço de memória
8
9
10     printf("Valor de num: %d \n",num);
11     printf("Endereço de memoria de num[0]: %d \n", &num[0]);
12
13     return 0;
14 }
```

```
Valor de num: 2358864
Endereço de memoria de num[0]: 2358864

Process returned 0 (0x0)   execution time: 0.000 s
Press any key to continue.
```



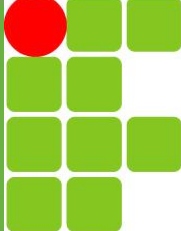
2.0 Vetores e Ponteiros

- Provando que o valor apontado por num = num[0]

```
teiro04.c  X
1  #include<stdio.h>
2
3  int main(){
4      int num[3];
5
6      // & Referencia um endereço de memória
7      // * Referencia o conteúdo de um endereço de memória
8
9      num[0] = 10;
10     num[1] = 20;
11     num[2] = 30;
12     printf("Valor apontado por num: %d \n", *num);
13     printf("Valor de num[0]: %d \n", num[0]);
14
15     return 0;
16 }
```

```
Valor apontado por num: 10
Valor de num[0]: 10

Process returned 0 (0x0)   ex
Press any key to continue.
```



2.0 Vetores e Ponteiros

- Provando que o valor apontado por `num+1` = `num[1]` ...

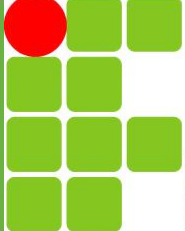
nteiro05.c X

```
1  #include<stdio.h>
2
3  int main() {
4      int num[3];
5
6      // & Referencia um endereço de memória
7      // * Referencia o conteúdo de um endereço
8
9      num[0] = 10;
10     num[1] = 20;
11     num[2] = 30;
12     printf("Valor apontado por num+1: %d \n", *(num+1));
13     printf("Valor de num[1]: %d \n", num[1]);
14     puts("\n"); // pulando uma linha
15     printf("Valor apontado por num+2: %d \n", *(num+2));
16     printf("Valor de num[2]: %d \n", num[2]);
17     return 0;
18 }
```

```
Valor apontado por num+1: 20
Valor de num[1]: 20

Valor apontado por num+2: 30
Valor de num[2]: 30

Process returned 0 (0x0)   exe
Press any key to continue.
```



2.0 Vetores e Ponteiros

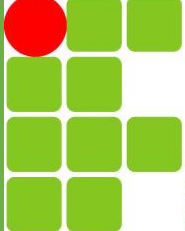
- Provando que o endereço de memória de `num[1] = num+1 ...`

ponteiro06.c X

```
1  #include<stdio.h>
2
3  int main(){
4      int num[3];
5
6      // & Referencia um endereço de memória
7      // * Referencia o conteúdo de um endereço de memória
8
9      num[0] = 10;
10     num[1] = 20;
11     num[2] = 30;
12     printf("Endereço de memória de num[1]: %d \n", &num[1]);
13     printf("Endereço de memória de num+1: %d \n", num+1);
14
15     return 0;
16 }
```

```
Endereço de memória de num[1]: 2358868
Endereço de memória de num+1: 2358868

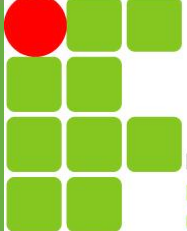
Process returned 0 (0x0)   execution time
Press any key to continue.
```



INSTITUTO FEDERAL
ESPÍRITO SANTO
Campus Cachoeiro de Itapemirim

Funções e Procedimentos

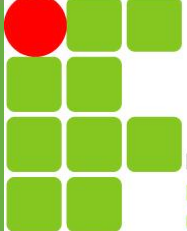
MODULARIZAÇÃO



INSTITUTO FEDERAL
ESPÍRITO SANTO
Campus Cachoeiro de Itapemirim

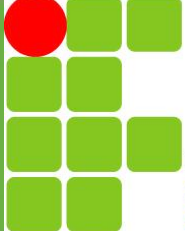
3.0 Modularização

- ▶ Ideologia de Função;
- ▶ Ideologia de Procedimento;
- ▶ Passagem de parâmetros:
 - ▶ Por valor;
 - ▶ Por referência;



3.0 Modularização

- ▶ Função
 - ▶ Executa uma ação e retorna valor.



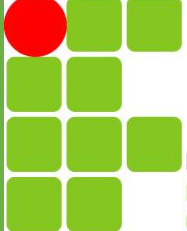
3.0 Mod

- ▶ Exemplo com e sem função
- ▶ Passagem por valor

```
dularizao01.c x
1  #include<stdio.h>
2
3  int main(){
4      int a,b,soma;
5
6      printf("Digite um numero: ");
7      scanf("%d",&a);
8      printf("Digite outro numero: ");
9      scanf("%d",&b);
10
11     soma = a + b;
12
13     printf("Soma: %d",soma);
14
15     return 0;
16 }
17
```

```
modularizao01.c x
1  #include<stdio.h>
2
3  // prototipo da funcao soma
4  int somar(int x, int y);
5
6  int main(){
7      int a,b,soma;
8
9      printf("Digite um numero: ");
10     scanf("%d",&a);
11     printf("Digite outro numero: ");
12     scanf("%d",&b);
13
14     soma = somar(a,b);
15
16     printf("Soma: %d",soma);
17
18     return 0;
19 }
20
21 int somar(int x, int y){
22     return x + y;
23 }
24
```

```
C:\!materialAula\I
Digite um numero: 3
Digite outro numero: 7
Soma: 10
Process returned 0 (0x0) e
Press any key to continue.
```



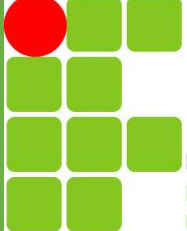
3.0 Modularização

► Função:

Passagem de parâmetro
por referência

```
C:\materialAula\Ita
Digite um numero: 5
Digite outro numero: 3
Soma: 8
Process returned 0 (0x0)   e
Press any key to continue.
_
```

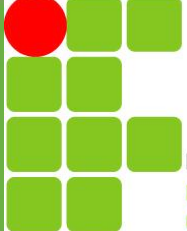
```
modularizao05.c x
1  #include<stdio.h>
2
3  // prototipo da funcao soma
4  int somar(int *x, int *y);
5
6  int main() {
7      int a,b,soma;
8
9      printf("Digite um numero: ");
10     scanf("%d",&a);
11     printf("Digite outro numero: ");
12     scanf("%d",&b);
13
14     soma = somar(&a,&b);
15
16     printf("Soma: %d",soma);
17
18     return 0;
19 }
20
21 int somar(int *x, int *y){
22     return (*x) + (*y);
23 }
24
```



3.0 Modularização

► Procedimento

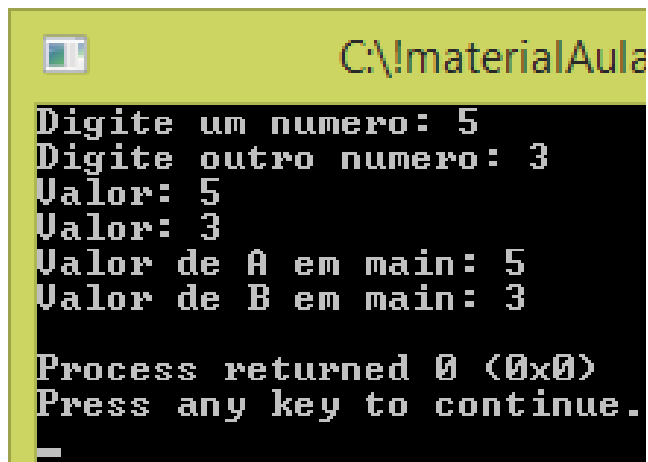
- Executa uma ação e NÃO retorna valor.



3.0 Modularização

► Procedimento:

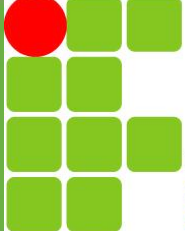
Passagem de parâmet
por valor



```
C:\!materialAula
Digite um numero: 5
Digite outro numero: 3
Valor: 5
Valor: 3
Valor de A em main: 5
Valor de B em main: 3

Process returned 0 (0x0)
Press any key to continue.
```

```
modularizao02.c X
1  #include<stdio.h>
2
3  // prototipos
4  void imprimirValor(int x);
5
6  int main(){
7
8      int a, b;
9
10     printf("Digite um numero: ");
11     scanf("%d", &a);
12
13     printf("Digite outro numero: ");
14     scanf("%d", &b);
15
16     imprimirValor(a);
17     imprimirValor(b);
18
19     // só para testar o valor de a
20     printf("Valor de A em main: %d\n", a);
21     printf("Valor de B em main: %d\n", b);
22     return 0;
23 }
24
25 void imprimirValor(int x){
26     printf("Valor: %d\n", x);
27 }
```



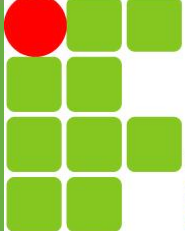
3.0 Modularização

- Procedimento:
Passagem de parâmet
por referência

```
C:\materialAula\I
Digite um numero: 5
Digite outro numero: 3
Valor: 5
Valor: 3
Valor de A em main: 25
Valor de B em main: 9

Process returned 0 (0x0)
Press any key to continue.
```

```
modularizao03.c x
1  #include<stdio.h>
2
3  // prototipos
4  void imprimirValor(int *x);
5
6  int main(){
7
8      int a, b;
9
10     printf("Digite um numero: ");
11     scanf("%d", &a);
12
13     printf("Digite outro numero: ");
14     scanf("%d", &b);
15
16     imprimirValor(&a);
17     imprimirValor(&b);
18
19     // só para testar o valor de a
20     printf("Valor de A em main: %d\n", a);
21     printf("Valor de B em main: %d\n", b);
22     return 0;
23 }
24
25 void imprimirValor(int *x){
26     printf("Valor: %d\n", *x);
27     (*x) = (*x) * (*x);
28 }
```



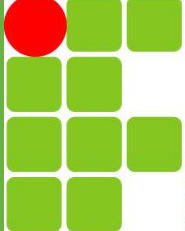
4.0 Treinando...

- ▶ Modularização retornando um tipo *char*

```
modularizao06.c X
1  #include<stdio.h>
2
3  // prototipo do módulo
4  char verificarConceito(float n);
5
6  int main(){
7      char conceito;
8      float nota;
9
10     printf("Digite a nota: ");
11     scanf("%f",&nota);
12     conceito = verificarConceito(nota);
13     printf("Nota: %.2f | Conceito: %c",nota, conceito);
14
15     return 0;
16 }
17
18 char verificarConceito(float n){
19     char conceito;
20     if (n >= 8){
21         conceito = 'A';
22     }else
23         if (n >= 6){
24             conceito = 'B';
25         }else{
26             conceito = 'C';
27         }
28     return conceito;
29 }
```

C:\material\Aula\IFES

```
Digite a nota: 8
Nota: 8.00 | Conceito: A
Process returned 0 (0x0)   exe
Press any key to continue.
```



4.0 Treinando...

- Passando um vetor com
Parâmetro. (Forma I)

modularizao10_a.c

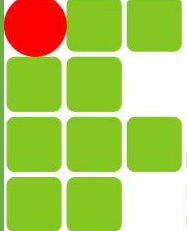
```
1  #include<stdio.h>
2
3  // prototipos
4  void imprimirNotas(float m[6], int n);
5
6  int main(){
7      int i;
8      float notas[6];
9      for (i = 0; i <= 5; i++){
10         printf("Digite a nota do aluno: ");
11         scanf("%f", &notas[i]);
12     }
13
14     imprimirNotas(notas, 6);
15
16     return 0;
17 }
18
19 void imprimirNotas(float m[6], int n){
20     int i;
21     for (i = 0; i < n; i++){
22         printf("Nota do aluno %d: %.2f\n", i, m[i]);
23     }
24 }
```



C:\!materialAula\IFES_20

```
Digite a nota do aluno: 34
Digite a nota do aluno: 43
Digite a nota do aluno: 2
Digite a nota do aluno: 56
Digite a nota do aluno: 78
Digite a nota do aluno: 98
Nota do aluno 0: 34.00
Nota do aluno 1: 43.00
Nota do aluno 2: 2.00
Nota do aluno 3: 56.00
Nota do aluno 4: 78.00
Nota do aluno 5: 98.00

Process returned 0 (0x0)
Press any key to continue.
```



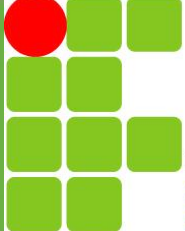
4.0 Treinando...

- ▶ Passando um vetor como Parâmetro. (Forma II)

```
modularizao10_b.c x
1  #include<stdio.h>
2
3  // prototipos
4  void imprimirNotas(float m[], int n);
5
6  int main(){
7      int i;
8      float notas[6];
9      for (i = 0; i <= 5; i++){
10         printf("Digite a nota do aluno: ");
11         scanf("%f", &notas[i]);
12     }
13
14     imprimirNotas(notas, 6);
15
16     return 0;
17 }
18
19 void imprimirNotas(float m[], int n){
20     int i;
21     for (i = 0; i < n; i++){
22         printf("Nota do aluno %d: %.2f\n", i, m[i]);
23     }
24 }
```



```
C:\materialAula\IFES_201
Digite a nota do aluno: 34
Digite a nota do aluno: 76
Digite a nota do aluno: 89
Digite a nota do aluno: 90
Digite a nota do aluno: 56
Digite a nota do aluno: 100
Nota do aluno 0: 34.00
Nota do aluno 1: 76.00
Nota do aluno 2: 89.00
Nota do aluno 3: 90.00
Nota do aluno 4: 56.00
Nota do aluno 5: 100.00
Process returned 0 (0x0)   e
Press any key to continue.
```

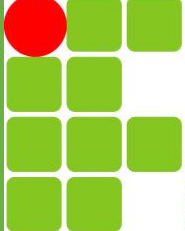


4.0 Treinando...

- Passando um vetor como Parâmetro. (Forma III)

```
C:\materialAula\I
Digite a nota do aluno: 65
Digite a nota do aluno: 78
Digite a nota do aluno: 98
Digite a nota do aluno: 45
Digite a nota do aluno: 67
Digite a nota do aluno: 100
Nota do aluno 0: 65.00
Nota do aluno 1: 78.00
Nota do aluno 2: 98.00
Nota do aluno 3: 45.00
Nota do aluno 4: 67.00
Nota do aluno 5: 100.00
```

```
modularizao09.c x
1  #include<stdio.h>
2
3  // prototipos
4  void imprimirNotas(float *m, int n);
5
6  int main(){
7      int i;
8      float notas[6];
9      for (i = 0; i <= 5; i++){
10         printf("Digite a nota do aluno: ");
11         scanf("%f", &notas[i]);
12     }
13
14     imprimirNotas(notas, 6);
15
16     return 0;
17 }
18
19 void imprimirNotas(float *m, int n){
20     int i;
21     for (i = 0; i < n; i++){
22         printf("Nota do aluno %d: %.2f\n", i, m[i]);
23     }
24 }
25
```

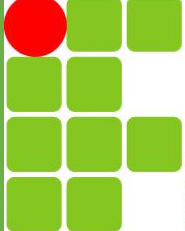


4.0 Treinando...

- Modularização retornando um conjunto de *char* (*string*)

```
modularizacao07.c X
1  #include<stdio.h>
2
3  // prototipo do módulo
4  char *verificarSituacao(float n);
5
6  int main(){
7      char *sit;
8      float nota;
9
10     printf("Digite a nota: ");
11     scanf("%f",&nota);
12     sit = verificarSituacao(nota);
13     printf("Nota: %.2f | Situacao: %s",nota, sit);
14
15     return 0;
16 }
17
18 char *verificarSituacao(float n){
19     char *sit;
20     if (n >= 60){
21         sit = "Aprovado";
22     }else
23         if (n >= 40){
24             sit = "Recuperacao";
25         }else{
26             sit = "Reprovado";
27         }
28     return sit;
29 }
```

```
C:\materialAula\IFES_2015
Digite a nota: 75
Nota: 75.00 | Situacao: Aprovado
Process returned 0 (0x0)   execution time: 0.000 sec
Press any key to continue.
```



INSTITUTO FEDERAL
ESPÍRITO SANTO
Campus Cachoeiro de Itapemirim

Dúvidas?

Flávio Izo

fizo@ifes.edu.br